CSE331 OPERATING SYSTEMS DESIGN FALL 2025 ASSIGNMENT II

Ece Düzgeç 20220702045

Synchronization with Semaphores Comparison Report

This report presents an analysis of two different approaches to Pet Shop service system using multithreaded using POSIX threads and multiprocess using fork() with shared memory. The system manages a service room where dogs and cats receive health care, ensuring that only one type can be in the room at any time, dogs have priority over cats, but cats don't starve. I tested both implementations with different room capacities and pet numbers to understand when to use threads or processes for synchronization problems.

The Problem

The pet shop has a service room with limited capacity (MAXIMUM). Dogs and cats arrive randomly and want service, but only one type of animal can be in the room at a time. Dogs have priority, but if we always serve dogs first, cats might wait forever causing starvation. The system needs to keep dogs and cats separated, give dogs priority, prevent cat starvation allow multiple same pets up to room capacity, handle pets arriving at random times and to prevent starvation the solution uses a batch quota mechanism that means after serving 5 dogs in a row and cats get priority.

Synchronization Logic

Both implementations use the same synchronization algorithm with three semaphores and one flag.

Semaphores Used:

mutex: A binary semaphore initialized to 1. It protects the critical sections where shared variables (like dogs_waiting or cats_in_room) are modified. Only one thread or process can hold the mutex at a time, ensuring that state updates are atomic and consistent.
dogs_queue: A counting semaphore initialized to 0. This acts as a waiting room for dogs. When a dog cannot enter the service room, it calls sem_wait on this semaphore and blocks. When a dog can enter, another pet calls sem_post on this semaphore to wake up one waiting dog.
cats_queue: A counting semaphore initialized to 0. This works the same way as dogs_queue but for cats. Cats wait here when they cannot enter, and get signaled when it's their turn.

The Cat Turn Flag:

cat_turn: An integer flag with values 0 or 1 that tracks whether it's currently the cats' turn to enter. When set to 0, dogs have priority. When set to 1, cats can enter even if dogs are waiting. This flag is important for preventing cat starvation.
When 5 dogs have been served consecutively and cats are waiting, cat_turn is set to 1
Cats can then enter regardless of dogs waiting.
When all waiting cats have been served, cat_turn is reset to 0, restoring dog priority.

Dogs need to wait if cats are in the room, wait if room is at capacity with dogs otherwise, enter immediately.
Cats need to wait if dogs are in the room, wait if any dogs are waiting and cat_turn is 0, wait if room is at capacity with cats otherwise enter.

Batch Quota:
A counter that tracks dogs served. After 5 dogs are served if cats are waiting the counter resets and cats get priority. This provides bounded waiting for cats with maximum wait time of $5 \times 1 = 5$ seconds. Without this, cats could starve indefinitely if dogs keep arriving.

Signaling When Leaving:
When a dog leaves: If room now empty check if 5 dogs served and cats waiting and signal cat and reset counter. Else if dogs waiting signal to dog for priority. Else if cats waiting to signal cat. If room is not empty but has space signal to another dog.
When a cat leaves: If room now empty prefer dogs to restore priority. If room not empty but has space and no dogs waiting signal to another cat.

1. Multithreaded Program using POSIX Threads (hw2a.c)
This program creates separate threads for each pet. All running within a single process. Threads share the same memory automatically, so they can all see the same variables for tracking room status.

The main program creates one thread for each dog and cat. All threads share global variables like dogs_in_room, cats_in_room, dogs_waiting, and cats_waiting. Three semaphores coordinate access: mutex protects the shared variables, dogs_queue blocks waiting dogs, and cats_queue blocks waiting cats.

pthread_create(): Creates a new thread for each pet
sem_wait(), **sem_post():** Lock and unlock access to shared data
pthread_join(): Main waits for all threads to finish
sem_init(): Initializes semaphores with pshared=0 for threads only

When a dog wants service, it first locks the mutex to safely check the room status. If cats are in the room or the room is full of dogs, the dog releases the mutex and waits on the dogs_queue semaphore. When signaled to wake up it reacquires the mutex, rechecks conditions and if it is clear enters the room. After getting service for 1 second, the dog leaves and signals the next pet based on priority and batch quota.

Cats follow a similar pattern but have more conditions to check. They must wait if dogs are in the room, dogs are waiting, the room is full of cats or the batch quota isn't full yet. This ensures dogs get priority while preventing cat starvation.

In this program, creating threads is very fast. Threads share memory automatically, so no complex setup needed. Context switching between threads is quick because they

share the same address space. Uses less memory since code and data are shared. The code is simpler and easier to debug with one process.

However, if one thread crashes, the entire program stops. All threads can access all memory which can cause a security problem. It cannot distribute threads across different computers. Easier to get bugs with shared memory if not careful with locks.

2.Multiprocess Program using fork() and Shared Memory (hw2b.c)
This program creates separate processes for each pet using fork(). Each process has its own private memory space so shared memory must be explicitly created.

The main program creates shared memory segments for the state variables and semaphores. Then it forks separate processes for each dog and cat. Each process can access the shared memory through pointers. The processes use the same synchronization logic as threads but access everything through pointers.

fork(): Creates a new process (like cloning the program)
shmget(): Creates shared memory space
shmat(): Attaches shared memory to process
sem_init(): Initializes semaphores with pshared=1 for processes
wait(): Parent waits for children to finish
shmdt(), shmctl(): Clean up shared memory

Shared memory structure holds all shared state: dogs_in_room, cats_in_room, dogs_waiting, cats_waiting, dogs_served, and MAXIMUM. This structure is in shared memory. Three semaphores also live in shared memory so all processes can use them.

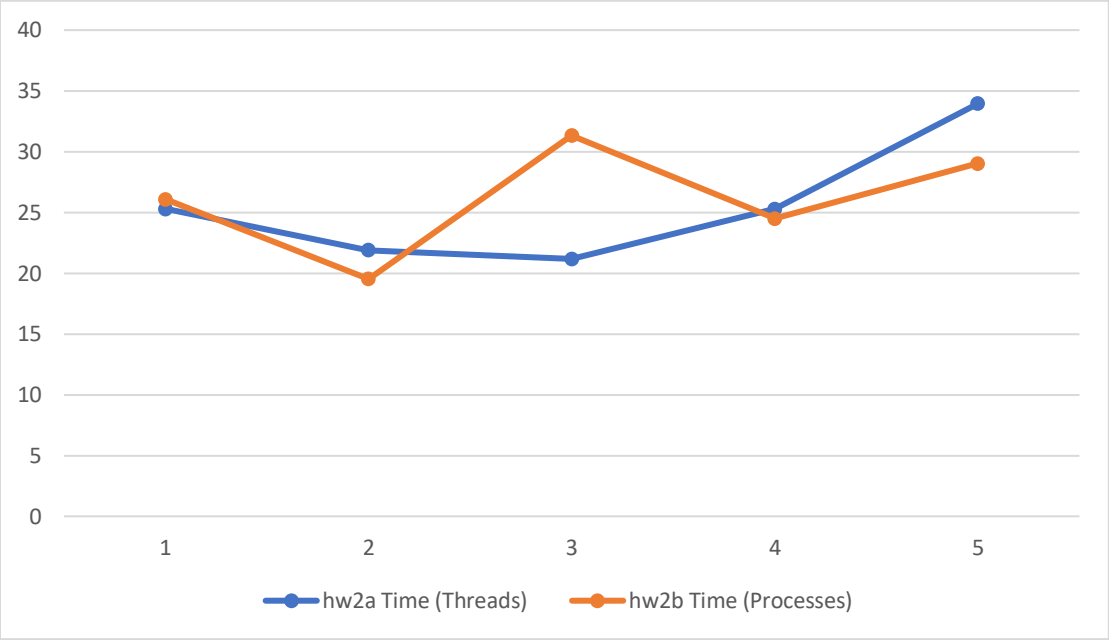
Instead of global variables, processes use pointers to shared memory: state->dogs_in_room instead of just dogs_in_room. After every printf, the code calls fflush(stdout) because each process has its own output buffer. The semaphores must be initialized with pshared=1 to work across processes. Cleanup requires explicitly releasing shared memory with shmdt() and shmctl().

When a process is created with fork() it gets a copy of the parent but then runs independently. The child process executes the pet arrival, wait if needed, getting service, leaving and then exits with exit(0). The parent process waits for all children using the wait() function.
It enables strong isolation means if one process crashes, others keep running and also better security with separate address spaces. It can run processes on different machines in distributed systems. More stable and robust to individual failures.

On the other hand, creating processes is much slower than the first program. Each process needs separate memory copies using more RAM. Context switching between processes is slower because the CPU must flush its cache. It must explicitly set up and clean up shared memory with multiple system calls. Harder to debug with many separate processes. More error prone with more system calls.

Performance Testing

Test Case	Room Capacity	Dogs	Cats	hw2a (Threads) Time	hw2b (Processes) Time
Test 1	1	10	10	25.293s	26.064s
Test 2	3	10	10	21.890s	19.522s
Test 3	3	10	5	21.176s	31.327s
Test 4	5	15	15	25.285s	24.496s
Test 5	10	20	20	33.945s	29.023s



The results show that performance varies depending on the test case. For the first test with capacity 1, threads were slightly faster. For tests with capacity 3, the results were mixed as threads were much faster in one case but processes were faster in another. For larger capacities like 5 and 10, processes actually performed better.
This difference happens because the execution time depends heavily on when pets randomly arrive. If pets happen to arrive in a pattern that lets the room fill up efficiently, the program runs faster. If they arrive in a way that causes more waiting, it takes longer. Since arrival is random each run can be different.

In all tests is the synchronization worked correctly every time. The batch quota mechanism successfully prevented cat starvation in all cases. After 5 dogs were served, all waiting cats got their turn before dogs resumed. The cat_turn flag was essential for this without it, cats would go back to sleep even after the quota was reached because dogs were still waiting.

Room capacity clearly affected performance with capacity 1, pets entered one at a time making it the slowest. With larger capacities like 10, multiple pets could be served simultaneously which significantly reduced total time. Both implementations handled all room sizes correctly without any deadlocks or issues.

So from these tests we can say that both threads and processes work well for this synchronization problem. Threads have simpler code because memory sharing is automatic, while processes provide better isolation if one crashes. For this pet shop problem where correctness matters most and both approaches successfully solved the synchronization challenge also the random arrival timing had more impact on speed than whether we used threads or processes.

Conclusion

Both implementations successfully solved the pet shop synchronization problem using semaphores. The test results showed that performance varied between the two approaches.The change of faster speed happened because the total execution time depends heavily on random arrival patterns of the pets, which can affect both implementations differently.

For the system work correctly we used three semaphores and one flag. The mutex semaphore protected shared data, the two queue semaphores made pets wait their turn, and the cat_turn flag prevented cat starvation. The flag makes sure that once cats get their turn all waiting cats are served before dog priority is restored.

Both implementations kept dogs and cats separated, gave dogs priority most of the time, and prevented cats from starving. The batch quota of 5 dogs worked well in all test cases. The testing showed that both versions handled different room sizes and pet numbers correctly without any deadlocks or race conditions.

For this synchronization problem both threads and processes work well. Threads have simpler code because memory sharing is automatic while processes require explicit shared memory. In practice the choice between them depends more on other factors like whether we need isolation where processes are better or simplicity where threads are better rather than performance since arrival has a bigger effect on execution time than the implementation.